TECHNISCHE UNIVERSITÄT WIEN

Make, Argument Parsing, Error Handling, Signals

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch, Peter Puschner

2026-10-16

Part I

Introduction to `make`

Compiling a program: Single source file

hello.c

```c
#include <stdio.h>

void say_hello(const char *name) {
    printf("Hello %s!\n", name)
}

int main(int argc, char *argv[]) {
    say_hello("Michael");
    return 0;
}
```

```
$ gcc -o hello hello.c
```

Compiling a program: Multiple source files

hello.h

```
1  void say_hello(const char *name);
```

hello.c

```
1  #include "hello.h"
2  #include <stdio.h>
3  void say_hello(const char *name) {
4      printf("Hello %s!\n", name);
5  }
```

main.c

```
1  #include "hello.h"
2  int main(int argc, char *argv[]) {
3      say_hello("Michael");
4      return 0;
5  }
```

Compile and link in one step:

```
1  $ gcc -o hello main.c hello.c
```

First compile source files into object files (***.o**):

```
1  $ gcc -c -o main.o main.c      # slow
2  $ gcc -c -o hello.o hello.c    # slow
```

Then link the object files together:

```
1  $ gcc -o hello main.o hello.o  # fast
```

This gives you an executable named `hello`

Compiling a program: Multiple source files

hello.h

```
1  void say_hello(const char *name);
```

hello.c

```
1  #include "hello.h"
2  #include <stdio.h>
3  void say_hello(const char *name) {
4      printf("Hello %s!\n", name);
5  }
```

main.c

```
1  #include "hello.h"
2  int main(int argc, char *argv[]) {
3      say_hello("Michael");
4      return 0;
5  }
```

After editing **hello.c**: no need to recompile **main.c**

```
1  $ gcc -c -o hello.o hello.c     # slow
2  $ gcc -o hello main.o hello.o   # fast
```

For large projects, compiling all source files takes a lot of time. Recompiling only changed files is much faster.

A Makefile is a file that stores Shell commands to build a software project

Purpose(s) of Makefiles:

- Split a complex build into sub-tasks
- Give simple names to long, complex commands
- Add flexibility (via variables)
- Abstract recurrent patterns

Structure of a Makefile rule


```
1 result: ingredients
2     recipe
```

- **result** (aka *target*): file that will be generated/updated by this rule
- **ingredients** (aka *dependencies*): files needed to create the target (if any)
- **recipe**: actual commands to create the target from the ingredients

- Comments may be inserted using #
- Very important: commands in the recipe **must** be indented with a **tab** (spaces won't work!)

Makefile example


```
1  hello: main.o hello.o
2      gcc -o hello main.o hello.o     # linking
3
4  main.o: main.c hello.h
5      gcc -c -o main.o main.c         # compiling
6
7  hello.o: hello.c hello.h
8      gcc -c -o hello.o hello.c       # compiling
```

- For each object file, dependencies include the corresponding source file and any header files needed by them
- Recipe for final executable only depends on object files

Make tool

- Example of a build automation tool (others: `ant` (Java), `cargo` (Rust), ...)
- Invocation: `make` + name of target
- By default, will load a file named **Makefile** in the current directory
- If no target is specified, the first target in the `Makefile` is built

How make works

```
1  hello:  main.o hello.o
2      gcc -o hello main.o hello.o
3
4  main.o:  main.c hello.h
5      gcc -c -o main.o main.c
6
7  hello.o:  hello.c hello.h
8      gcc -c -o hello.o hello.c
```

Working directory

```
Makefile
main.c
hello.h
hello.c
```

```
$ make
```

- First target: **hello**
  - Does not exist, so it must be built
  - Dependencies: **main.o** and **hello.o**
  - Neither exists, so they must be built

How make works

```
1  hello:  main.o hello.o
2      gcc -o hello main.o hello.o
3
4  main.o:  main.c hello.h
5      gcc -c -o main.o main.c
6
7  hello.o:  hello.c hello.h
8      gcc -c -o hello.o hello.c
```

Working directory

```
Makefile   main.o
main.c
hello.h
hello.c
```

```
$ make
```

- First target: **hello**
  - Does not exist, so it must be built
  - Dependencies: **main.o** and **hello.o**
  - Neither exists, so they must be built
- 2nd target: **main.o**
  - Dependencies: **main.c** and **hello.h**
  - Both exist, so we can execute the recipe

How make works

```
1  hello:  main.o hello.o
2      gcc -o hello main.o hello.o
3
4  main.o:  main.c hello.h
5      gcc -c -o main.o main.c
6
7  hello.o:  hello.c hello.h
8      gcc -c -o hello.o hello.c
```

Working directory

```
Makefile   main.o
main.c     hello.o
hello.h
hello.c
```

```
$ make
```

- First target: **hello**
  - Does not exist, so it must be built
  - Dependencies: **main.o** and **hello.o**
  - Neither exists, so they must be built
- 2nd target: **main.o**
  - Dependencies: **main.c** and **hello.h**
  - Both exist, so we can execute the recipe
- 3rd target: **hello.o** (same as above)

How make works


```
1  hello:  main.o hello.o
2      gcc -o hello main.o hello.o
3
4  main.o:  main.c hello.h
5      gcc -c -o main.o main.c
6
7  hello.o:  hello.c hello.h
8      gcc -c -o hello.o hello.c
```

Working directory

```
Makefile   main.o
main.c     hello.o
hello.h    hello
hello.c
```

```
$ make
```

- First target: **hello**
  - Does not exist, so it must be built
  - Dependencies: **main.o** and **hello.o**
  - Neither exists, so they must be built
- 2nd target: **main.o**
  - Dependencies: **main.c** and **hello.h**
  - Both exist, so we can execute the recipe
- 3rd target: **hello.o** (same as above)
- Now all dependencies of **hello** are up to date: we can execute the recipe

Incremental builds


```
1  hello:  main.o hello.o
2      gcc -o hello main.o hello.o
3
4  main.o: main.c hello.h
5      gcc -c -o main.o main.c
6
7  hello.o:  hello.c hello.h
8      gcc -c -o hello.o hello.c
```

Working directory

`Makefile`	(7.00)	`main.o`	(7.30)
`main.c`	(7.00)	`hello.o`	(7.30)
`hello.h`	(7.00)	`hello`	(7.30)
`hello.c`	**(8.00)**		

OS tracks modification times of files

We edit only **hello.c** at 8.00

We run `make` again at 8.15

Incremental builds


```
1  hello:  main.o hello.o
2      gcc -o hello main.o hello.o
3
4  main.o: main.c hello.h
5      gcc -c -o main.o main.c
6
7  hello.o:  hello.c hello.h
8      gcc -c -o hello.o hello.c
```

Working directory

`Makefile`	(7.00)	`main.o`	(7.30)
`main.c`	(7.00)	`hello.o`	**(8.15)**
`hello.h`	(7.00)	`hello`	(7.30)
`hello.c`	(8.00)		

OS tracks modification times of files

We edit only **hello.c** at 8.00

We run `make` again at 8.15

- First target: **hello**
  - Depends on **hello.o**
    - Which depends on **hello.c**
    - This is newer than **hello.o**!
  - So we must rebuild **hello.o**

Incremental builds


```
1  hello:  main.o hello.o
2      gcc -o hello main.o hello.o
3
4  main.o: main.c hello.h
5      gcc -c -o main.o main.c
6
7  hello.o:  hello.c hello.h
8      gcc -c -o hello.o hello.c
```

Working directory

`Makefile`	(7.00)	`main.o`	(7.30)
`main.c`	(7.00)	`hello.o`	(8.15)
`hello.h`	(7.00)	`hello`	**(8.15)**
`hello.c`	(8.00)		

OS tracks modification times of files

We edit only **hello.c** at 8.00

We run `make` again at 8.15

- First target: **hello**
  - Depends on **hello.o**
    - Which depends on **hello.c**
    - This is newer than **hello.o**!
  - So we must rebuild **hello.o**
- Now **hello.o** is newer than **hello**
- Rebuild **hello** as well
- We did not recompile **main.c** at all

Variables

- Define with `VAR = value`
- Use with `$(VAR)`
- Can override from command line: `make VAR=value`
- Common in C projects: `CC` (compiler), `CFLAGS` (compiler flags), `LDFLAGS` (linker flags)

```
1  CC = gcc
2  CFLAGS = -std=c99 -Wall
3
4  hello: main.o hello.o
5      $(CC) $(CFLAGS) -o hello main.o hello.o
6
7  main.o: main.c hello.h
8      $(CC) $(CFLAGS) -c -o main.o main.c
9
10 hello.o: hello.c hello.h
11     $(CC) $(CFLAGS) -c -o hello.o hello.c
```

Patterns and Automatic variables

Pattern rules specify recipes for a whole class of targets
For instance: `%.o` for all object files

- We need special variables to retrieve target and dependency names in the recipe:
  - `$@`: target
  - `$<`: first dependency
  - `$^`: all dependencies
- We can always add explicit dependencies (e.g., for `.h` files)

```
1  CC = gcc
2  CFLAGS = -std=c99 -Wall
3
4  hello: main.o hello.o
5      $(CC) $(CFLAGS) -o $@ $^
6
7  %.o: %.c
8      $(CC) $(CFLAGS) -c -o $@ $<
9
10 hello.o: hello.c hello.h
11 main.o: main.c hello.h
```

Phony targets

Targets that do not correspond to actual files, only used to give a name to a rule

- Declared with `.PHONY`
- "Standard" rules:
  - **all**: conventionally 1st recipe, says what to build
  - **clean**: remove all built files
  - **install**: copy built files to system directories (just FYI, we won't need this)

```
1  CC = gcc
2  CFLAGS = -std=c99 -Wall
3  .PHONY: all clean
4
5  all: hello
6
7  hello: main.o hello.o
8      $(CC) $(CFLAGS) -o $@ $^
9
10 %.o: %.c
11     $(CC) $(CFLAGS) -c -o $@ $<
12
13 hello.o: hello.c hello.h
14 main.o: main.c hello.h
15
16 clean:
17     rm -rf *.o hello
```

Makefile: Complete Example


```
1  CC = gcc
2  # Avoid a few warnings related to C99/POSIX vs GNU implementations
3  DEFS = -D_DEFAULT_SOURCE -D_POSIX_C_SOURCE=200809L
4  # Enable all warnings, debug symbols, C99 standard, and detailed checks
5  CFLAGS = -Wall -g -std=c99 -pedantic $(DEFS)
6  OBJECTS = main.o hello.o
7
8  .PHONY: all clean
9  all: hello
10
11 hello: $(OBJECTS)
12     $(CC) -o $@ $^
13
14 %.o: %.c
15     $(CC) $(CFLAGS) -c -o $@ $<
16
17 main.o: main.c hello.h
18 hello.o: hello.c hello.h
19
20 clean:
21     rm -rf *.o hello
```

Part II

Argument Parsing

Shell Commands


```
1  gcc -c -o hello.o hello.c
2  make clean
3  ls -l -a /usr/bin
4  cd ~/Documents
5  echo Hello World
```

- program name, or path to the executable
- options that control the behaviour of the command
  - (usually `-` and a single character, or `--` and a longer name)
  - Some option may take option-arguments (e.g., `-o hello.o` in the first example)
- positional arguments on which the command is executed
  (e.g., paths to files/directories, or just strings, as in `echo`)

Pass Arguments to `main`


```c
int main (int argc, char **argv);
int main (int argc, char *argv[]);  /* equivalent */
```

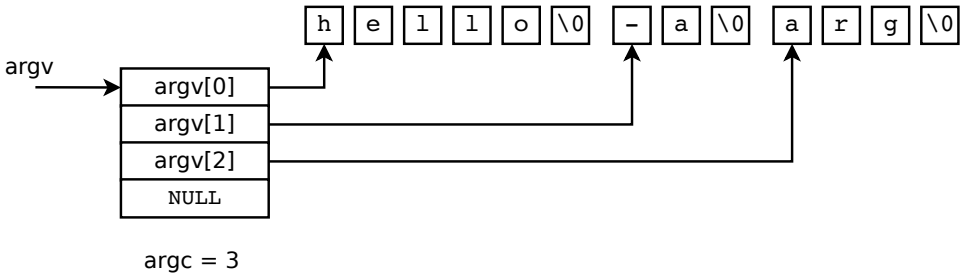
argc Number of elements in `argv`

argv Array of command line arguments (`argv[0]`...`argv[argc-1]`)

- `argv[0]`: usually the program name or path to the executable
- `argv[argc]` is always `NULL` according to the standard
- Value of `argc`: number of command line arguments + 1

Why is argv a char**?

- `hello -a arg`



Examples of argc and argv

- `make`

  argc: 1 argv[0]: `make`

- `gcc -c main.c hello.c`

  argc: 4 argv[0]: `gcc`
  argv[1]: `-c`
  argv[2]: `main.c`
  argv[3]: `hello.c`

- `echo "-f -o hello.txt"`

  argc: 2 argv[0]: `echo`
  argv[1]: `-f -o hello.txt`

Handle Options: `getopt(3)`


```
int getopt(int argc, char *const argv[], const char *optstring);
```

- Standard function (<unistd.h>) to parse command line options[1]
- `optstring` defines which options are recognized
- Example: `"a:bc::"`

  `a:` Option `-a` that requires an argument
  `b` Option `-b` without an argument
  `c::` Option `-c` with optional argument
- Order is usually irrelevant

[1] getopt_long() supports also long option names

getopt - Usage

Each call to `getopt()` parses one option from `argv`

- If it returns -1, there are no more options to parse
- If it returns '?', an error was detected (invalid option)
- Otherwise, it returns the option character
- If the option had an argument, it sets a global pointer `optarg` to point to it
- Call `getopt()` within a loop, break when it returns -1

After the loop (`getopt()` returned -1), check for positional arguments

- `getopt()` tracks the index of the next argument in a global variable `optind`
- If `argc = optind`, there are no positional arguments
- Otherwise, positional arguments start at `argv[optind]` until `argv[argc-1]`

getopt - Example


```
1  char *a_arg = NULL; int opt_o = 0; int c;
2  while ( (c = getopt(argc, argv, "a:o")) != -1 ) {
3    switch ( c ) {
4      case 'a': a_arg = optarg;            break;
5      case 'o': opt_o++;                   break;
6      case '?': /* invalid option */       break;
7      default:  /* should not occur */     break;
8    }
9  }
```

```
10 if ( a_arg == NULL ) { /* option 'a' did not occur */ }
11 if ( opt_o > 1 )     { /* option 'o' occurs more than once */ }
12 if ( optind < argc ) { /* there are positional arguments */ }
```

Documents correct ways of calling a program, e.g.,

```
Usage: hello [-o] file...
```

(This syntax is also used in man pages, so-called Synopsis)

Mandatory arguments:	`hello -o`
Optional arguments: `[]`	`hello -a optarg [-o] arg1`
Alternative options: `[x|y]`	`hello [-a optarg | -o] arg1`
Alternative required arguments: `{x|y}`	`hello {-a optarg | -o} arg1`
Conditionally valid options:	`hello [-a optarg [-o]] arg1`
One or more occurrence of a positional argument	`hello -a optarg file...`

Usage Message: Example


```
1  void usage(int retcode) {
2    fprintf(stderr,"Usage: hello [-a file] [-o] file\n", myprog);
3    exit(retcode);
4  }
5
6  int main (int argc, char *argv[]) {
7    int c = -1;
8    while ( (c = getopt(argc, argv, "h..")) != -1 ) {
9      switch ( c ) {    case 'h': usage(EXIT_SUCCESS); break;
10                       case '?': usage(EXIT_FAILURE); break;
11                       ... /* handle other options */         }
12   }
13
14   if ( /* user did not correctly call program */ ) { usage(EXIT_FAILURE); }
15   /* ... */
16 }
```

Part III

Error Handling

Error Handling: `strerror(3)`


```
char* strerror(int errnum);
```

- Many functions return -1 or `NULL` on error and set the global variable
- They also write a more detailed error code to `errno` (defined in <errno.h>)
- `strerror()` returns a human-readable description of error code

If an error is reported

- In general: you will need to program some recovery strategy
- In this course: simply print a meaningful error message to `stderr` and terminate the program properly

Error Handling

Meaningful error messages

- Which program? (`argv[0]`)
- What problem? (e.g., "`fopen failed`")
- Cause? (`strerror(errno)`)
- Print to `stderr`

Proper termination

1. Free all resources (memory, files, ...)
2. Call `exit(EXIT_FAILURE)`

   (See `man 3 exit`)

```
1  FILE *file;
2  if ( (file = fopen("input.txt", "r")) == NULL ) {
3    fprintf(stderr, "[%s] ERROR: fopen failed: %s\n", argv[0], strerror(errno));
4    exit(EXIT_FAILURE);
5  }
```

```
[./prog] ERROR: fopen failed: No such file or directory
```

Part IV

Signals

Signals

- A signal is a message sent to a process to notify an event
- In Unix, signals are standardised (Number + name starting with `SIG`)
- Some signals have predefined handling:
  - Ignore (e.g., `SIGCHLD`)
  - Terminate process (e.g., `SIGTERM`)
  - Produce core dump (e.g., `SIGSEGV`)
- For most signals, you can define custom handling
- `SIGSTOP` and `SIGKILL` cannot be handled or ignored
- Signals may interrupt certain system/library calls
  - `errno == EINTR`
  - In this case, usually just retry

Configuring Signal Actions: `sigaction(2)`


```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

- `signum`: number of the signal to handle
- `act`: pointer to new configuration (leave `NULL` to keep unchanged)
- `oldact`: pointer to store old configuration (leave `NULL` to ignore)

Returns 0 on success, -1 on error ($\rightarrow$ `errno`)

```
1  #include <signal.h>
2
3  struct sigaction {
4    void     (*sa_handler)(int);
5    sigset_t   sa_mask;
6    int        sa_flags;
7    ...
8  };
```

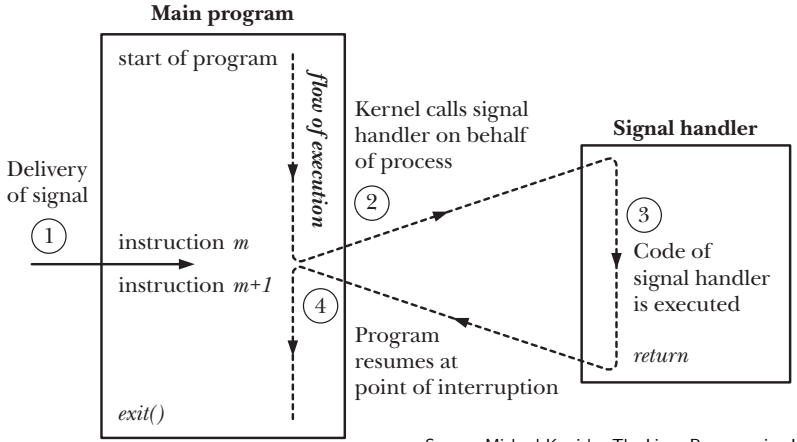
- `sa_handler` Address of signal handler, or `SIG_DFL` (default handler), or `SIG_IGN` (ignore)
- `sa_mask` Signals that should be blocked while the handler for this signal is executed
- `sa_flags` Additional options (see manual)

Signal Configuration

Example

```
1  void handle_signal(int signal) { ... }
2
3  int main(int argc, char *argv[]) {
4      struct sigaction sa;        // Notice we cannot use dynamic memory
5      memset(&sa, 0, sizeof(sa)); // initialize sa to 0 (see 'man 3 memset')
6
7      sa.sa_handler = handle_signal;  // Points sa_handler to our function
8
9      sigaction(SIGINT, &sa, NULL);
10     ...
11 }
```

Procedure



Source: Michael Kerrisk - The Linux Programming Interface, p. 399

Writing a Signal Handler

Function prototype: `void handle_signal(int signal);`

- Should be as simple as possible
- Be careful with the usage of static variables (consistency)
- Handler can also be interrupted by another signal
  - You can use signal masks to block signals,
  - But you cannot block `SIGKILL` and `SIGSTOP`
- In the signal handler, use async-signal-safe functions only.

Async-signal-safe functions

From `man 7 signal-safety`

An async-signal-safe function is one that can be safely called from within a signal handler. Many functions are not async-signal-safe. [...] In general, a function is async-signal-safe either because it is reentrant[2] or because it is atomic with respect to signals (i.e., its execution can't be interrupted by a signal handler).

Not async-signal-safe (examples)

malloc, free, printf, exit, fopen, ...

Some async-signal-safe functions (full list: `man 7 signal-safety`)

abort, _exit, write, sigaction, ...

[2] i.e., can be interrupted in the middle of its execution and safely called again ("re-entered")

Signal Handler

Example 1

```c
void handle_signal(int signal) {
    _exit(1); // IMMEDIATE termination (as opposed to exit())
}
```

- But what if we want to clean up before exiting?
- ... What if we don't want to exit immediately, or at all?
- Idea: just use handler to set a global flag, then check it in `main`

Signal Handler

Example 2

```
1  volatile sig_atomic_t quit = 0;
2  void handle_signal(int signal) {
3      quit = 1;
4  }
5
6  int main(int argc, char **argv) {
7      ... // Set up signal handler
8
9      while (!quit) { /* main application loop */ }
10     ... // Additional cleanup
11     return 0;
12 }
```

- Why `volatile`? To avoid optimizations
- Why `sig_atomic_t`? So signals cannot interfere with read/write of the flag

Material

- GNU Make Manual

  `http://www.gnu.org/software/make/manual/html_node/`
- Utility Conventions for Argument Syntax

  `http://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap12.html`
- The GNU C Library Reference Manual, Ch. 25 (signals)

  `https://sourceware.org/glibc/manual/latest/html_node/Signal-Handling.html`

Manual pages

Built-in documentation in most Unix systems. Command: `man <name>`
Only documentation available in the exams, so practice reading them!
Examples from today:

- `man make`
- `man 1 getopt`, `man 3 getopt`, `man 3 strerror`,
- `man 2 sigaction`, `man 7 signal`, `man 7 signal-safety`
- And of course `man man`!